

Start coding or [generate](#) with AI.

```
documents = [
    # Sports
    "The local soccer team won their championship match last night in a",
    "Basketball analytics are becoming increasingly sophisticated, infl",
    "The Olympics committee announced new regulations for athlete eligi",
    "Running marathons requires immense discipline and physical enduran",
    "Extreme sports like skydiving and whitewater rafting continue to g",

    # Politics
    "The new bill on climate change is currently under debate in the pa",
    "International leaders gathered for a summit to discuss global econ",
    "Local elections are approaching, with several candidates campaigni",
    "A recent political scandal involving campaign finance has led to c",
    "Geopolitical tensions are rising in the East, prompting urgent dip",

    # Health
    "Advances in medical research have led to new treatments for chroni",
    "Public health officials are urging citizens to adopt healthier lif",
    "Mental health awareness campaigns are gaining traction, encouragin",
    "The importance of regular vaccinations for preventing infectious d",
    "Nutrition science reveals the complex interplay between diet and g",

    # Technology
    "Artificial intelligence is transforming industries from healthcare",
    "The latest smartphone models feature enhanced cameras and faster p",
    "Cybersecurity threats are evolving rapidly, requiring constant upd",
    "Quantum computing research is making significant strides, promisin",
    "Renewable energy technologies, such as solar and wind power, are b",
]

print(f"Generated {len(documents)} sample documents.")
```

Generated 20 sample documents.

```
import re # For regular expressions, useful in text cleaning
import string # For string manipulation, like punctuation removal
import nltk # Natural Language Toolkit for various NLP tasks

# Scikit-learn for numerical representation and similarity calculations
from sklearn.feature_extraction.text import CountVectorizer, TfidfVecto
from sklearn.metrics.pairwise import cosine_similarity # For calculatin

# NLTK components for WordNet-based semantic similarity and lemmatizati
from nltk.corpus import wordnet # For accessing WordNet lexical databas
from nltk.stem import WordNetLemmatizer # For lemmatization, reducing w
```

```
# Download necessary NLTK data files. This needs to be done once.
nltk.download('punkt', quiet=True) # For tokenization (splitting text i
nltk.download('stopwords', quiet=True) # For a list of common words to
nltk.download('wordnet', quiet=True) # For WordNet database, used in le
nltk.download('omw-1.4', quiet=True) # Open Multilingual Wordnet, often

print("Required libraries imported and NLTK data downloaded.")
```

Required libraries imported and NLTK data downloaded.

```
print("--- Sample Documents ---")
for i, doc in enumerate(documents[:3]):
    print(f"Document {i+1}: {doc[:100]}...") # Print first 100 characte
print("-----")
```

```
--- Sample Documents ---
Document 1: The local soccer team won their championship match last night in a t
Document 2: Basketball analytics are becoming increasingly sophisticated, influe
Document 3: The Olympics committee announced new regulations for athlete eligibi
-----
```

```
from nltk.corpus import stopwords, wordnet
from nltk.stem import WordNetLemmatizer
from nltk.tag import pos_tag
import re
import string
import nltk # Ensure nltk is imported for download calls

# Download necessary NLTK data files here if not already done in a prev
nltk.download('punkt') # Ensure punkt tokenizer is available
nltk.download('punkt_tab') # Explicitly download punkt_tab as per error
nltk.download('averaged_perceptron_tagger_eng') # Required for pos_tag,
nltk.download('stopwords') # For a list of common words to remove
nltk.download('wordnet') # For WordNet database, used in lemmatization
nltk.download('omw-1.4') # Open Multilingual Wordnet, often needed with

# Initialize NLTK resources once for efficiency
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

# Helper function to convert NLTK's POS tag to WordNet's format for lem
def get_wordnet_pos(tag):
    if tag.startswith('J'):
        return wordnet.ADJ
    elif tag.startswith('V'):
        return wordnet.VERB
    elif tag.startswith('N'):
        return wordnet.NOUN
    elif tag.startswith('R'):
        return wordnet.ADV
```

```

else:
    return wordnet.NOUN # Default to noun if not found

def preprocess_text(text, lemmatize=True):
    """
    Performs text preprocessing including lowercasing, removing punctua
    tokenization, stopwords removal, and optional lemmatization.
    """
    # 1a. Convert text to lowercase
    text = text.lower()

    # 1b. Remove punctuation
    # Use re.sub to replace any character in string.punctuation with a
    text = re.sub(f'[{re.escape(string.punctuation)}]', '', text)

    # 1c. Remove numbers
    text = re.sub(r'\d+', '', text)

    # 1d. Tokenize the text
    tokens = nltk.word_tokenize(text)

    # 1e. Remove stopwords
    tokens = [word for word in tokens if word not in stop_words]

    # 1f. Optional: Lemmatization
    if lemmatize:
        lemmatized_tokens = []
        # Use pos_tag to get part-of-speech tags for better lemmatizati
        pos_tags = pos_tag(tokens)
        for word, tag in pos_tags:
            w_pos = get_wordnet_pos(tag)
            lemmatized_tokens.append(lemmatizer.lemmatize(word, pos=w_p
        tokens = lemmatized_tokens

    # 1g. Join the preprocessed tokens back into a single string
    return ' '.join(tokens)

# 2. Apply the preprocess_text function to all documents
preprocessed_documents = [preprocess_text(doc) for doc in documents]

# 3. Print the original first document and its preprocessed version
print("--- Preprocessing Demonstration ---")
print(f"Original Document 1:\n{documents[0]}\n")
print(f"Preprocessed Document 1:\n{preprocessed_documents[0]}")
print("-----")

```

```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]   /root/nltk_data...

```

```
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
--- Preprocessing Demonstration ---
Original Document 1:
The local soccer team won their championship match last night in a thrilling over

Preprocessed Document 1:
local soccer team championship match last night thrill overtime victory fan cele
-----
```

```
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd

# 1. Calculate cosine similarity matrix using TF-IDF vectors
cosine_sim_matrix = cosine_similarity(tfidf_matrix)

# 2. Print the entire cosine similarity matrix
print("--- Cosine Similarity Matrix (TF-IDF based) ---")
# Convert to DataFrame for better readability, especially with large ma
sim_df = pd.DataFrame(cosine_sim_matrix)
print(sim_df)
print("-----")

# 3. Interpret at least 5 pairs of documents
print("\n--- Interpretation of Document Similarity ---")

# Helper function to get document text
def get_doc_text(doc_index):
    return documents[doc_index]

# Pair 1: Document 0 (Sports) vs Document 1 (Sports)
# Expected: High similarity as both are sports related
doc1_idx, doc2_idx = 0, 1
score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text(doc1_idx)[:50]}'...) v
print(f"Interpretation: Both documents are about sports (soccer and bas

# Pair 2: Document 5 (Politics) vs Document 6 (Politics)
# Expected: High similarity as both are politics related
doc1_idx, doc2_idx = 5, 6
score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text(doc1_idx)[:50]}'...) v
print(f"Interpretation: These documents discuss political topics (clima

# Pair 3: Document 0 (Sports) vs Document 5 (Politics)
# Expected: Low similarity as they are from different categories
doc1_idx, doc2_idx = 0, 5
```

```

score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text(doc1_idx)[:50]}...') v
print(f"Interpretation: These documents are from completely different t

# Pair 4: Document 10 (Health) vs Document 11 (Health)
# Expected: High similarity as both are health related
doc1_idx, doc2_idx = 10, 11
score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text(doc1_idx)[:50]}...') v
print(f"Interpretation: Both documents are related to health (medical r

# Pair 5: Document 15 (Technology) vs Document 0 (Sports)
# Expected: Low similarity as they are from different categories
doc1_idx, doc2_idx = 15, 0
score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text(doc1_idx)[:50]}...') v
print(f"Interpretation: This pair compares technology (AI) with sports

```

--- Cosine Similarity Matrix (TF-IDF based) ---

	0	1	2	3	4	5	6	\
0	1.000000	0.046109	0.044642	0.000000	0.000000	0.000000	0.000000	
1	0.046109	1.000000	0.047589	0.046830	0.000000	0.000000	0.000000	
2	0.044642	0.047589	1.000000	0.000000	0.000000	0.028542	0.000000	
3	0.000000	0.046830	0.000000	1.000000	0.000000	0.000000	0.000000	
4	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	
5	0.000000	0.000000	0.028542	0.000000	0.000000	1.000000	0.000000	
6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	
7	0.048649	0.000000	0.000000	0.000000	0.000000	0.096495	0.000000	
8	0.000000	0.000000	0.000000	0.000000	0.000000	0.051698	0.000000	
9	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.052420	
10	0.000000	0.000000	0.027912	0.000000	0.000000	0.029568	0.000000	
11	0.000000	0.000000	0.000000	0.000000	0.000000	0.039350	0.000000	
12	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.052135	
13	0.000000	0.000000	0.000000	0.000000	0.054094	0.000000	0.045991	
14	0.000000	0.040337	0.000000	0.000000	0.000000	0.050809	0.000000	
15	0.000000	0.000000	0.028710	0.000000	0.049427	0.030413	0.000000	
16	0.045708	0.000000	0.027587	0.000000	0.000000	0.029223	0.000000	
17	0.000000	0.094659	0.045824	0.090186	0.000000	0.048542	0.000000	
18	0.043532	0.000000	0.000000	0.000000	0.000000	0.047594	0.000000	
19	0.000000	0.043065	0.000000	0.000000	0.000000	0.000000	0.043832	

	7	8	9	10	11	12	13	\
0	0.048649	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
1	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
2	0.000000	0.000000	0.000000	0.027912	0.000000	0.000000	0.000000	
3	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
4	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.054094	
5	0.096495	0.051698	0.000000	0.029568	0.039350	0.000000	0.000000	
6	0.000000	0.000000	0.052420	0.000000	0.000000	0.052135	0.045991	
7	1.000000	0.043303	0.054339	0.000000	0.040480	0.152090	0.000000	
8	0.043303	1.000000	0.000000	0.050558	0.000000	0.042771	0.000000	
9	0.054339	0.000000	1.000000	0.000000	0.049374	0.000000	0.000000	
10	0.000000	0.050558	0.000000	1.000000	0.038482	0.000000	0.045321	
11	0.040480	0.000000	0.049374	0.038482	1.000000	0.033552	0.132873	

```

12 0.152090 0.042771 0.000000 0.000000 0.033552 1.000000 0.039515
13 0.000000 0.000000 0.000000 0.045321 0.132873 0.039515 1.000000
14 0.000000 0.000000 0.000000 0.000000 0.079943 0.035274 0.038217
15 0.000000 0.052003 0.000000 0.029742 0.000000 0.000000 0.000000
16 0.000000 0.000000 0.000000 0.028578 0.000000 0.000000 0.000000
17 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
18 0.000000 0.000000 0.000000 0.046544 0.000000 0.000000 0.000000
19 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.048623

```

```

          14          15          16          17          18          19
0  0.000000 0.000000 0.045708 0.000000 0.043532 0.000000
1  0.040337 0.000000 0.000000 0.094659 0.000000 0.043065
2  0.000000 0.028710 0.027587 0.045824 0.000000 0.000000
3  0.000000 0.000000 0.000000 0.090186 0.000000 0.000000
4  0.000000 0.049427 0.000000 0.000000 0.000000 0.000000
5  0.050809 0.030413 0.029223 0.048542 0.047594 0.000000
6  0.000000 0.000000 0.000000 0.000000 0.000000 0.043832
7  0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
8  0.000000 0.052003 0.000000 0.000000 0.000000 0.000000
9  0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
10 0.000000 0.029742 0.028578 0.000000 0.046544 0.000000

```

✓ Compute Jaccard Similarity

Subtask:

Calculate the Jaccard similarity for all possible pairs of tokenized text documents.

Compare the results with the cosine similarity scores. Interpret at least 5 pairs, specifically focusing on how overlapping words influence the Jaccard similarity score.

Reasoning: I need to define a `jaccard_similarity` function, iterate through all pairs of `preprocessed_documents` (splitting them into tokens), calculate the Jaccard similarity for each pair, store the results in a matrix, display the matrix, and then interpret 5 pairs comparing Jaccard and Cosine similarities.

```

import numpy as np
import pandas as pd

# 1. Define the Jaccard similarity function
def jaccard_similarity(doc1_tokens, doc2_tokens):
    """
    Calculates the Jaccard Similarity between two tokenized documents.
    Inputs are lists of tokens (words).
    """
    set1 = set(doc1_tokens)
    set2 = set(doc2_tokens)

    intersection = len(set1.intersection(set2))

```

```

union = len(set1.union(set2))

if union == 0:
    return 0.0
return intersection / union

# Prepare tokenized documents for Jaccard calculation
# The preprocessed_documents are strings, so we need to split them into lists
tokenized_preprocessed_documents = [doc.split() for doc in preprocessed_documents]

# 2. Create an empty NumPy array for the Jaccard similarity matrix
num_documents = len(tokenized_preprocessed_documents)
jaccard_sim_matrix = np.zeros((num_documents, num_documents))

# 3. and 4. Iterate through all possible pairs and calculate Jaccard similarity
for i in range(num_documents):
    for j in range(num_documents):
        if i <= j: # Calculate for each unique pair once and fill symmetrically
            score = jaccard_similarity(tokenized_preprocessed_documents[i], tok
            jaccard_sim_matrix[i, j] = score
            jaccard_sim_matrix[j, i] = score # Symmetric matrix

# 5. Print the entire Jaccard similarity matrix as a Pandas DataFrame
print("--- Jaccard Similarity Matrix ---")
jaccard_df = pd.DataFrame(jaccard_sim_matrix)
print(jaccard_df)
print("-----")

# 6. Interpret at least 5 document pairs
print("\n--- Interpretation of Jaccard Similarity ---")

# Helper function to get original document text
def get_doc_text_jaccard(doc_index):
    return documents[doc_index]

# Pair 1: Document 0 (Sports) vs Document 1 (Sports)
# Expected: Relatively high Jaccard score due to shared sports vocabulary.
doc1_idx, doc2_idx = 0, 1
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_jaccard(doc1_idx)[:50]}...') \n
print(f"  Jaccard Similarity: {j_score:.4f}")
print(f"  Cosine Similarity: {c_score:.4f}")
print(f"  Interpretation: Both documents are from the 'Sports' category. The Jac

# Pair 2: Document 5 (Politics) vs Document 6 (Politics)
# Expected: Moderate to high Jaccard due to shared political terms.
doc1_idx, doc2_idx = 5, 6
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_jaccard(doc1_idx)[:50]}...') \n

```

```

print(f" Jaccard Similarity: {j_score:.4f}")
print(f" Cosine Similarity: {c_score:.4f}")
print(" Interpretation: These documents are both about 'Politics'. While there

# Pair 3: Document 0 (Sports) vs Document 5 (Politics)
# Expected: Low Jaccard due to different topics and minimal overlapping words.
doc1_idx, doc2_idx = 0, 5
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_jaccard(doc1_idx)[:50]}...') v
print(f" Jaccard Similarity: {j_score:.4f}")
print(f" Cosine Similarity: {c_score:.4f}")
print(" Interpretation: Comparing 'Sports' and 'Politics' documents, both Jacc

# Pair 4: Document 10 (Health) vs Document 11 (Health)
# Expected: High Jaccard due to shared health-related terms.
doc1_idx, doc2_idx = 10, 11
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_jaccard(doc1_idx)[:50]}...') v
print(f" Jaccard Similarity: {j_score:.4f}")
print(f" Cosine Similarity: {c_score:.4f}")
print(" Interpretation: Both documents cover 'Health' topics. The Jaccard scor

# Pair 5: Document 15 (Technology) vs Document 16 (Technology)
# Expected: High Jaccard due to shared technology terms.
doc1_idx, doc2_idx = 15, 16
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_jaccard(doc1_idx)[:50]}...') v
print(f" Jaccard Similarity: {j_score:.4f}")
print(f" Cosine Similarity: {c_score:.4f}")
print(" Interpretation: These 'Technology' documents (AI, smartphone) share te

```

--- Jaccard Similarity Matrix ---

	0	1	2	3	4	5	6	\
0	1.000000	0.028571	0.027778	0.000000	0.000000	0.000000	0.000000	
1	0.028571	1.000000	0.029412	0.029412	0.000000	0.000000	0.000000	
2	0.027778	0.029412	1.000000	0.000000	0.000000	0.029412	0.000000	
3	0.000000	0.029412	0.000000	1.000000	0.000000	0.000000	0.000000	
4	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	
5	0.000000	0.000000	0.029412	0.000000	0.000000	1.000000	0.000000	
6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	
7	0.029412	0.000000	0.000000	0.000000	0.000000	0.064516	0.000000	
8	0.000000	0.000000	0.000000	0.000000	0.000000	0.031250	0.000000	
9	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.033333	
10	0.000000	0.000000	0.029412	0.000000	0.000000	0.030303	0.000000	
11	0.000000	0.000000	0.000000	0.000000	0.000000	0.028571	0.000000	
12	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.032258	
13	0.000000	0.000000	0.000000	0.000000	0.033333	0.000000	0.034483	
14	0.000000	0.030303	0.000000	0.000000	0.000000	0.030303	0.000000	
15	0.000000	0.000000	0.030303	0.000000	0.031250	0.031250	0.000000	

```

16 0.028571 0.000000 0.029412 0.000000 0.000000 0.030303 0.000000
17 0.000000 0.031250 0.030303 0.062500 0.000000 0.031250 0.000000
18 0.027027 0.000000 0.000000 0.000000 0.000000 0.028571 0.000000
19 0.000000 0.032258 0.000000 0.000000 0.000000 0.000000 0.033333

```

```

      7      8      9      10      11      12      13 \
0 0.029412 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
1 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
2 0.000000 0.000000 0.000000 0.029412 0.000000 0.000000 0.000000
3 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
4 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.033333
5 0.064516 0.031250 0.000000 0.030303 0.028571 0.000000 0.000000
6 0.000000 0.000000 0.033333 0.000000 0.000000 0.032258 0.034483
7 1.000000 0.032258 0.033333 0.000000 0.029412 0.103448 0.000000
8 0.032258 1.000000 0.000000 0.031250 0.000000 0.032258 0.000000
9 0.033333 0.000000 1.000000 0.000000 0.030303 0.000000 0.000000
10 0.000000 0.031250 0.000000 1.000000 0.028571 0.000000 0.033333
11 0.029412 0.000000 0.030303 0.028571 1.000000 0.029412 0.100000
12 0.103448 0.032258 0.000000 0.000000 0.029412 1.000000 0.034483
13 0.000000 0.000000 0.000000 0.033333 0.100000 0.034483 1.000000
14 0.000000 0.000000 0.000000 0.000000 0.058824 0.031250 0.033333
15 0.000000 0.032258 0.000000 0.031250 0.000000 0.000000 0.000000
16 0.000000 0.000000 0.000000 0.030303 0.000000 0.000000 0.000000
17 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
18 0.000000 0.000000 0.000000 0.028571 0.000000 0.000000 0.000000
19 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.035714

```

```

      14      15      16      17      18      19
0 0.000000 0.000000 0.028571 0.000000 0.027027 0.000000
1 0.030303 0.000000 0.000000 0.031250 0.000000 0.032258
2 0.000000 0.030303 0.029412 0.030303 0.000000 0.000000
3 0.000000 0.000000 0.000000 0.062500 0.000000 0.000000
4 0.000000 0.031250 0.000000 0.000000 0.000000 0.000000
5 0.030303 0.031250 0.030303 0.031250 0.028571 0.000000
6 0.000000 0.000000 0.000000 0.000000 0.000000 0.033333
7 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
8 0.000000 0.032258 0.000000 0.000000 0.000000 0.000000
9 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
10 0.000000 0.031250 0.030303 0.000000 0.028571 0.000000

```

```

import numpy as np
import pandas as pd
from nltk.corpus import wordnet
from nltk.tag import pos_tag

# 1. Define a helper function to convert NLTK POS tags to WordNet POS tags
def get_wordnet_pos_wn(word, tag):
    # First, try to use the WordNetLemmatizer's get_wordnet_pos if it's available
    # or a similar logic. For direct use with wordnet.synsets(), often a general
    # The previous `get_wordnet_pos` function was for lemmatizer. This one is for
    if tag.startswith('J'):
        return wordnet.ADJ
    elif tag.startswith('V'):
        return wordnet.VERB

```

```

        elif tag.startswith('N'):
            return wordnet.NOUN
        elif tag.startswith('R'):
            return wordnet.ADV
        else:
            return wordnet.NOUN # Default to noun if not found, or None to let syns

print("Helper function `get_wordnet_pos_wn` defined.")

Helper function `get_wordnet_pos_wn` defined.

```

Reasoning: The next step is to define the `document_wordnet_similarity` function, which will calculate the semantic similarity between two documents using WordNet's `wup_similarity`. This function will utilize the previously defined `get_wordnet_pos_wn` to get appropriate WordNet POS tags for synset retrieval and will handle averaging best scores for directional similarity.

Task

```

import numpy as np
import pandas as pd
from nltk.corpus import wordnet
from nltk.tag import pos_tag
from nltk.stem import WordNetLemmatizer # Needed for lemmatization within the func

# Re-initialize lemmatizer and stop_words for clarity, although already initialized
lemmatizer = WordNetLemmatizer()
stop_words = set(nltk.corpus.stopwords.words('english'))

# Re-define get_wordnet_pos_wn from previous cell for self-contained execution if n
def get_wordnet_pos_wn(tag):
    if tag.startswith('J'):
        return wordnet.ADJ
    elif tag.startswith('V'):
        return wordnet.VERB
    elif tag.startswith('N'):
        return wordnet.NOUN
    elif tag.startswith('R'):
        return wordnet.ADV
    else:
        return None # Return None if no specific WordNet tag is found, allowing syn

```

```

def document_wordnet_similarity(doc1_text, doc2_text):
    """
    Computes the WordNet Wu-Palmer similarity between two documents.
    This function processes the text to get tokens, POS tags them,
    and then calculates the average best Wu-Palmer similarity between
    words in doc1 and words in doc2.
    """

    # Preprocessing for WordNet similarity (can be slightly different from TF-IDF/)
    # For WordNet, we might want to keep some words that are stopwords but have sense
    # For now, we'll reuse the lemmatized and stopwords-removed tokens, but need to
    # from the original preprocessed_documents strings to get back the individual v

    # Tokenize and POS tag each document
    # We will use nltk.word_tokenize and pos_tag
    # The preprocessed_documents list already contains lemmatized and stopwords-rem
    # so we'll re-tokenize these for POS tagging.
    tokens1 = nltk.word_tokenize(doc1_text)
    tokens2 = nltk.word_tokenize(doc2_text)

    # Filter out non-alphabetic words and empty strings after tokenization
    tokens1 = [word for word in tokens1 if word.isalpha()]
    tokens2 = [word for word in tokens2 if word.isalpha()]

    if not tokens1 or not tokens2:
        return 0.0 # No words to compare

    pos_tagged_tokens1 = pos_tag(tokens1)
    pos_tagged_tokens2 = pos_tag(tokens2)

    similarity_scores = []

    # Calculate directional similarity from doc1 to doc2
    for word1, tag1 in pos_tagged_tokens1:
        best_similarity_for_word1 = 0.0
        wn_pos1 = get_wordnet_pos_wn(tag1)
        if wn_pos1:
            synsets1 = wordnet.synsets(word1, pos=wn_pos1)
        else:
            synsets1 = wordnet.synsets(word1) # Fallback if specific POS not found

        if not synsets1:
            continue # No synsets for word1

```

```

    for syn1 in synsets1:
        for word2, tag2 in pos_tagged_tokens2:
            wn_pos2 = get_wordnet_pos_wn(tag2)
            if wn_pos2:
                synsets2 = wordnet.synsets(word2, pos=wn_pos2)
            else:
                synsets2 = wordnet.synsets(word2) # Fallback

            if not synsets2:
                continue # No synsets for word2

            for syn2 in synsets2:
                similarity = syn1.wup_similarity(syn2)
                if similarity is not None and similarity > best_similarity_for_word1:
                    best_similarity_for_word1 = similarity
            if best_similarity_for_word1 > 0: # Only add if some similarity was found
                similarity_scores.append(best_similarity_for_word1)

    if not similarity_scores:
        return 0.0 # No common words or similarities found

    return np.mean(similarity_scores)

# 2. Compute the WordNet similarity matrix
num_documents = len(preprocessed_documents)
wordnet_sim_matrix = np.zeros((num_documents, num_documents))

print("Calculating WordNet semantic similarity (this may take some time)...")

for i in range(num_documents):
    for j in range(num_documents):
        if i <= j: # Calculate for each unique pair once and fill symmetrically
            score = document_wordnet_similarity(preprocessed_documents[i], preprocessed_documents[j])
            wordnet_sim_matrix[i, j] = score
            wordnet_sim_matrix[j, i] = score # Symmetric matrix

# 3. Print the matrix
print("\n--- WordNet Semantic Similarity Matrix (Wu-Palmer based) ---")
wordnet_df = pd.DataFrame(wordnet_sim_matrix)
print(wordnet_df)
print("-----")

```

```

# 4. Interpret at least 5 document pairs
print("\n--- Interpretation of WordNet Semantic Similarity ---")

# Helper function to get original document text
def get_doc_text_wordnet(doc_index):
    return documents[doc_index]

# Pair 1: Document 0 (Sports) vs Document 1 (Sports)
# Expected: High similarity, as 'soccer team' and 'basketball analytics' are related
doc1_idx, doc2_idx = 0, 1
wn_score = wordnet_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_wordnet(doc1_idx)[:50]}...' ) vs (Doc {doc2_idx+1}: '{get_doc_text_wordnet(doc2_idx)[:50]}...')")
print(f"  WordNet Similarity: {wn_score:.4f}")
print(f"  Cosine Similarity: {c_score:.4f}")
print(f"  Jaccard Similarity: {j_score:.4f}")
print("  Interpretation: Both are sports documents. WordNet provides a higher similarity score than Cosine and Jaccard.")

# Pair 2: Document 5 (Politics) vs Document 6 (Politics)
# Expected: Higher WordNet than Jaccard/Cosine, as 'climate change' and 'international relations' are related
doc1_idx, doc2_idx = 5, 6
wn_score = wordnet_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_wordnet(doc1_idx)[:50]}...' ) vs (Doc {doc2_idx+1}: '{get_doc_text_wordnet(doc2_idx)[:50]}...')")
print(f"  WordNet Similarity: {wn_score:.4f}")
print(f"  Cosine Similarity: {c_score:.4f}")
print(f"  Jaccard Similarity: {j_score:.4f}")
print("  Interpretation: These political documents have some conceptual overlap. WordNet provides a higher similarity score than Cosine and Jaccard.")

# Pair 3: Document 0 (Sports) vs Document 5 (Politics)
# Expected: Low similarity across all methods, as they are distinct topics.
doc1_idx, doc2_idx = 0, 5
wn_score = wordnet_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_wordnet(doc1_idx)[:50]}...' ) vs (Doc {doc2_idx+1}: '{get_doc_text_wordnet(doc2_idx)[:50]}...')")
print(f"  WordNet Similarity: {wn_score:.4f}")
print(f"  Cosine Similarity: {c_score:.4f}")
print(f"  Jaccard Similarity: {j_score:.4f}")
print("  Interpretation: As expected for documents from different topics, all three methods show low similarity scores.")

```

```
# Pair 4: Document 10 (Health) vs Document 11 (Health)
# Expected: High WordNet similarity, as both health documents contain semantically
doc1_idx, doc2_idx = 10, 11
wn_score = wordnet_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_wordnet(doc1_idx)[:50]}...' ) vs (Doc {doc2_idx+1}: '{get_doc_text_wordnet(doc2_idx)[:50]}...')")
print(f"  WordNet Similarity: {wn_score:.4f}")
print(f"  Cosine Similarity: {c_score:.4f}")
print(f"  Jaccard Similarity: {j_score:.4f}")
print("  Interpretation: These health-related documents show high similarity across all metrics")

# Pair 5: Document 15 (Technology - AI) vs Document 18 (Technology - Quantum Computing)
# Expected: Moderate to high WordNet similarity, as they are both subtopics of technology
doc1_idx, doc2_idx = 15, 18
wn_score = wordnet_sim_matrix[doc1_idx, doc2_idx]
c_score = cosine_sim_matrix[doc1_idx, doc2_idx]
j_score = jaccard_sim_matrix[doc1_idx, doc2_idx]
print(f"\nPair (Doc {doc1_idx+1}: '{get_doc_text_wordnet(doc1_idx)[:50]}...' ) vs (Doc {doc2_idx+1}: '{get_doc_text_wordnet(doc2_idx)[:50]}...')")
print(f"  WordNet Similarity: {wn_score:.4f}")
print(f"  Cosine Similarity: {c_score:.4f}")
print(f"  Jaccard Similarity: {j_score:.4f}")
print("  Interpretation: Both documents are about different aspects of 'Technology' but share some common terms, resulting in moderate similarity")
```